

What is PyCRAM?

PyCRAM is the Python 3 re-implementation of [CRAM](#). PyCRAM is a toolbox for designing, implementing and deploying software on autonomous robots. The framework provides various tools and libraries for aiding in robot software development as well as geometric reasoning and fast simulation mechanisms to develop cognition-enabled control programs that achieve high levels of robot autonomy.

PyCRAM is developed in Python with support for the ROS middleware which is used for communication with different software components as well as the robot.

This framework is tested with Ubuntu 20.04, ROS Noetic and Python 3.8

Simple Demonstration

PyCRAM allows the execution of the same high-level plan on different robot platforms. Below you can see an example of this where the plan is executed on the PR2 and the IAI's Boxy.

The plan that both robots execute is a relatively simple pick and place plan:

- They start at the world origin
- park their arms
- move to the counter
- observe the object
- pickup the object
- move to the kitchen island
- place the object
- move to the world origin

The code for this plan can be seen below.

```
from pycram.worlds.bullet_world import BulletWorld
from pycram.world_concepts.world_object import Object
from pycram.process_module import simulated_robot
from pycram.designators.motion_designator import *
from pycram.designators.location_designator import *
from pycram.designators.action_designator import *
from pycram.designators.object_designator import *
from pycram.datastructures.enums import ObjectType, Arms, Grasp,
WorldMode, TorsoState

world = BulletWorld(WorldMode.GUI)
kitchen = Object("kitchen", ObjectType.ENVIRONMENT, "kitchen.urdf")
robot = Object("pr2", ObjectType.ROBOT, "pr2.urdf")
cereal = Object("cereal", ObjectType.BREAKFAST_CEREAL,
"breakfast_cereal.stl", pose=Pose([1.4, 1, 0.95]))
```

```

cereal_desig = ObjectDesignatorDescription(names=["cereal"])
kitchen_desig = ObjectDesignatorDescription(names=["kitchen"])
robot_desig = ObjectDesignatorDescription(names=["pr2"]).resolve()

with simulated_robot:
    ParkArmsAction([Arms.BOTH]).resolve().perform()

    MoveTorsoAction([TorsoState.HIGH]).resolve().perform()

    pickup_pose = CostmapLocation(target=cereal_desig.resolve(),
    reachable_for=robot_desig).resolve()
    pickup_arm = pickup_pose.reachable_arms[0]

    NavigateAction(target_locations=[pickup_pose.pose]).resolve().perform()

    PickupAction(object_designator_description=cereal_desig,
    arms=[pickup_arm], grasps=[Grasp.FRONT]).resolve().perform()

    ParkArmsAction([Arms.BOTH]).resolve().perform()

    place_island = SemanticCostmapLocation("kitchen_island_surface",
    kitchen_desig.resolve(), cereal_desig.resolve()).resolve()

    place_stand = CostmapLocation(place_island.pose,
    reachable_for=robot_desig, reachable_arm=pickup_arm).resolve()

    NavigateAction(target_locations=[place_stand.pose]).resolve().perform()

    PlaceAction(cereal_desig, target_locations=[place_island.pose],
    arms=[pickup_arm]).resolve().perform()

    ParkArmsAction([Arms.BOTH]).resolve().perform()

world.exit()

```